파일 시스템과 파일 입출력

유닉스 프로그래밍

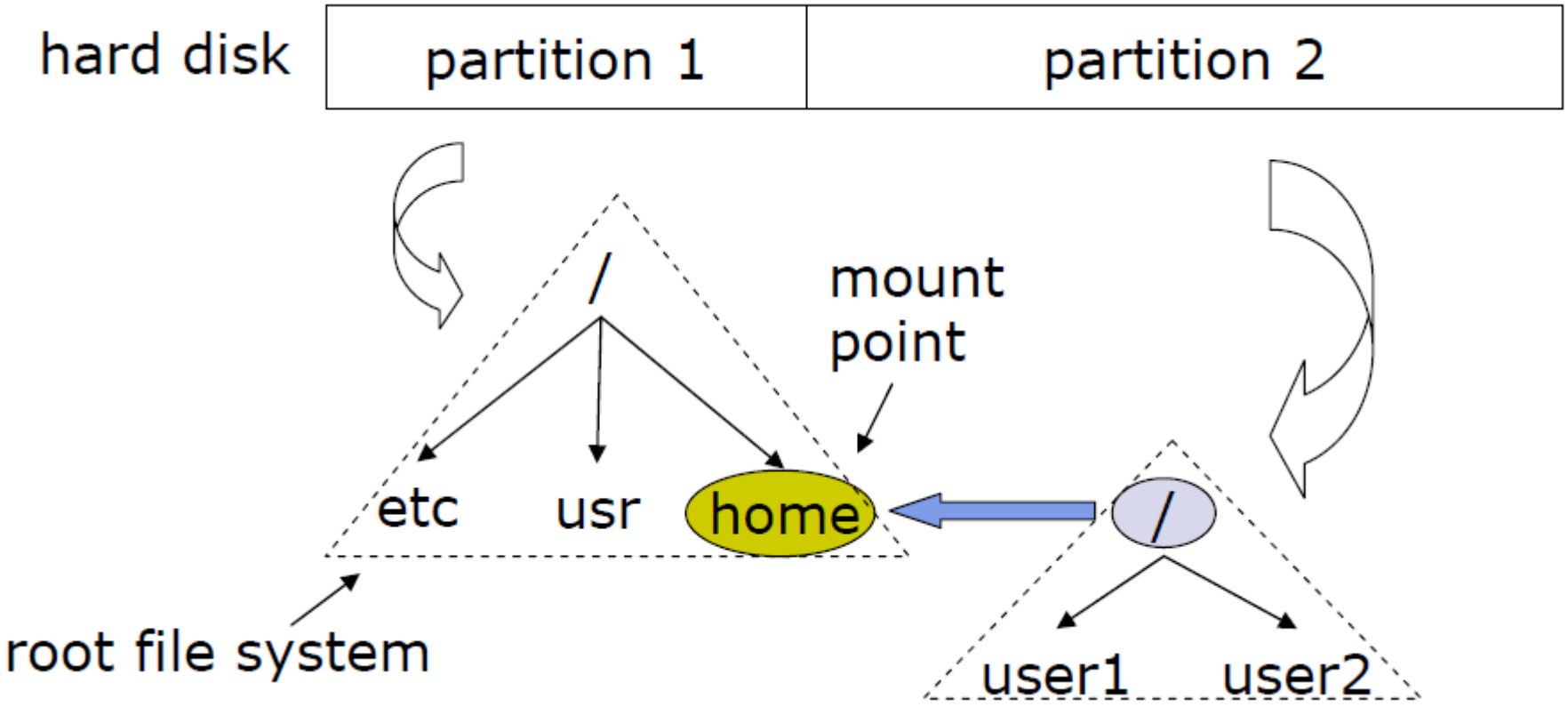
파일 시스템

- **파일**
  - 일반적 정의
    - 데이터의 모임으로 보조 기억장치에 저장
  - 리눅스에서의 정의
    - 위의 정의를 포함
    - Stream of bytes
    - 많은 객체 (장치, 네트워크 연결 등)를 동일한 인터페이스로 접근 가능

- **파일 시스템**
  - 보조기억장치의 일정 영역에 파일 읽기, 쓰기, 저장 및 삭제를 담당하는 시스템
  - 파일에 이름 부여
  - 디스크의 파일 위치 기록

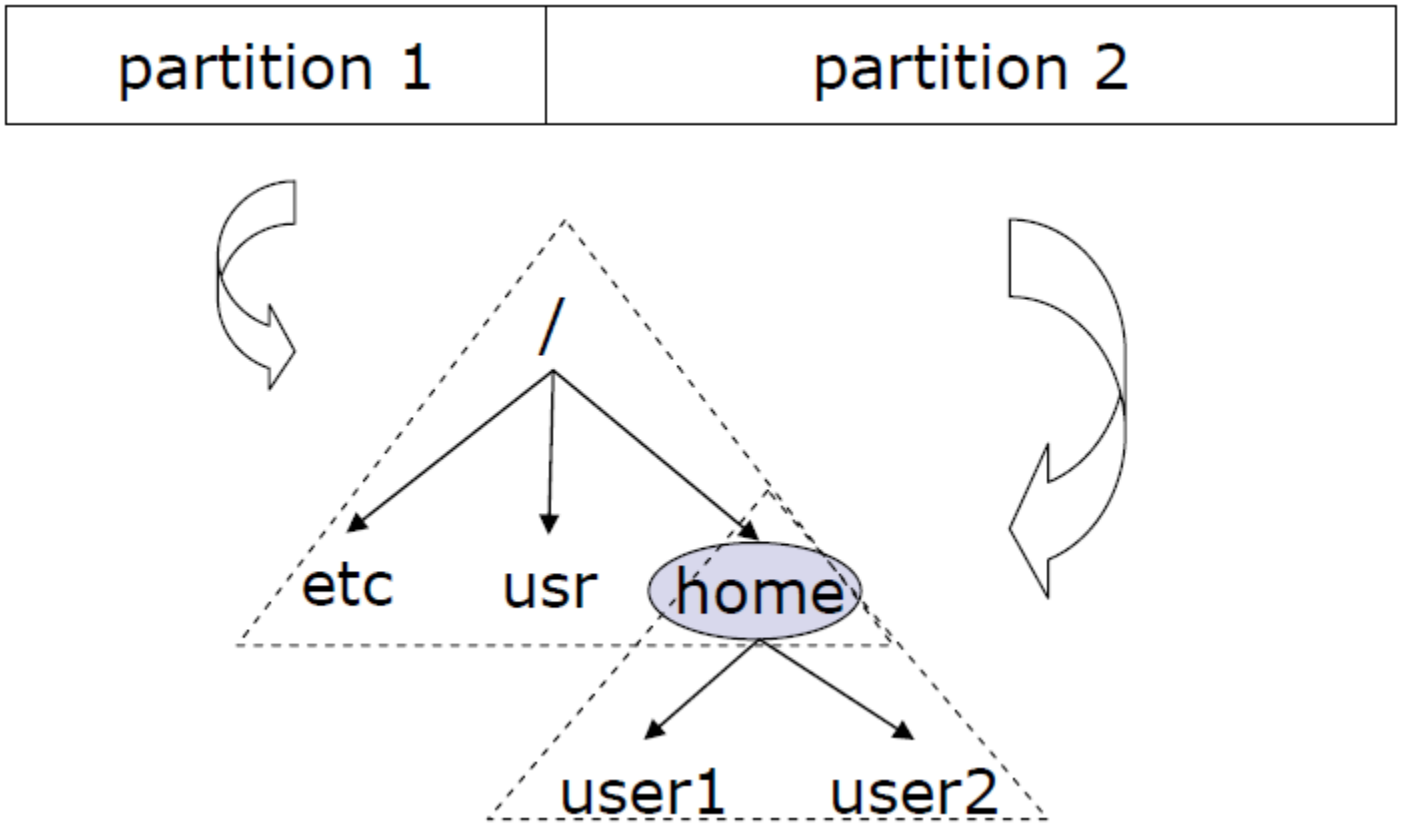
파일 시스템

- **마운트 (Mount)**
  - 여러 개의 파일 시스템을 하나의 디렉토리 트리에 접목

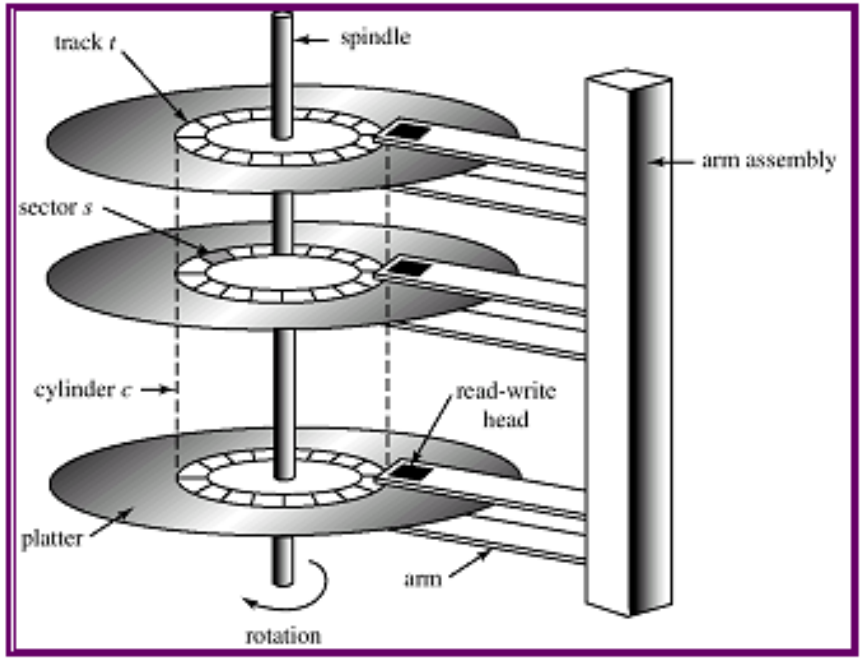


파일 시스템

- 마운트 결과



디스크의 물리 구조

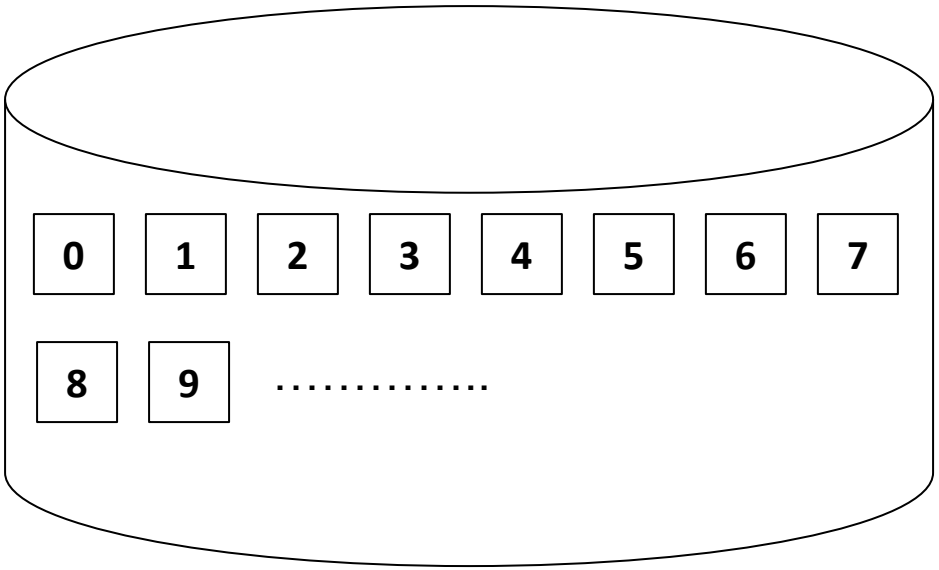




- **디스크 접근 방법**
  - Sector addressing: head(surface), track, sector
  - Head move to appropriate track: 탐색 시간 (Seek time)
  - Wait for the sector to appear under the head: 회전 지연 시간 (Rotational delay)
  - Read/write sector: 데이터 전달 시간 (Data transmission time)

디스크의 논리 구조

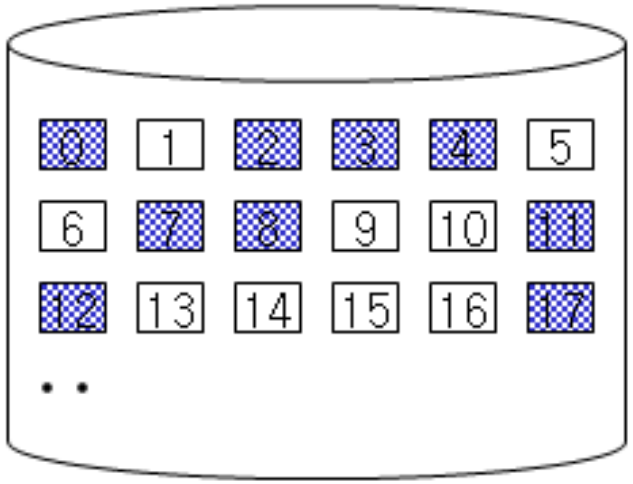
- **논리적 관점 (Linux 입장에서의 디스크 구조)**
  - 디스크는 디스크 블록의 집합
  - 디스크 블록의 크기는 일반적으로 1~8KB (2의 멱승으로 커짐)



블록 번호를 섹터 번호로 변환하는 작업은
컨트롤러나 디스크 드라이버에 의해 수행

디스크의 블록 관리 (1/4)

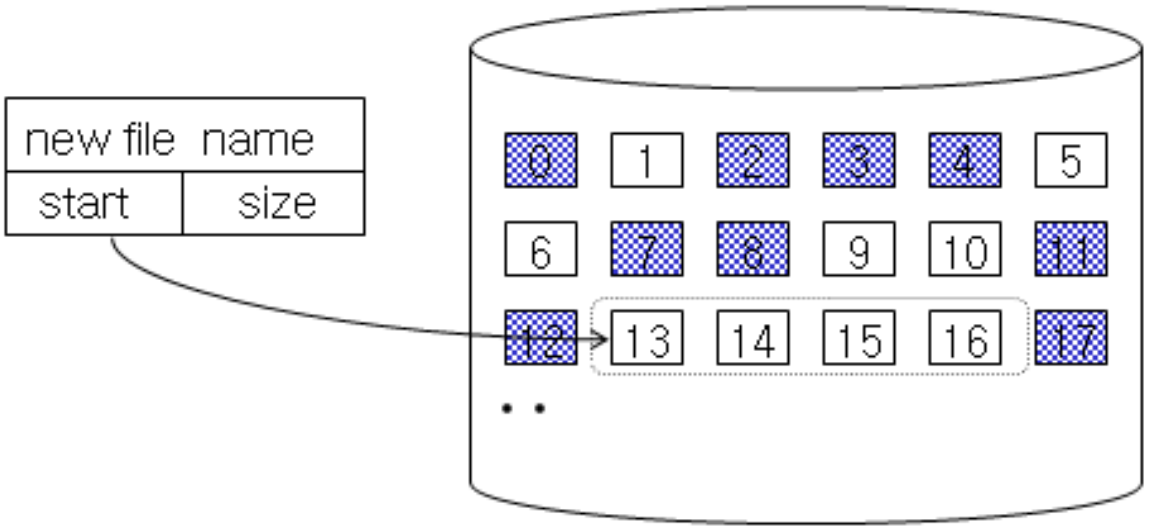
- **시나리오**
  - 새로운 14KB 크기의 파일을 디스크에 쓰려고 함
  - 디스크 블록의 크기는 4KB라 가정 (따라서 4개의 블록이 필요)
  - 아래 그림에서 빗금 친 부분은 이미 사용중인 (데이터를 가지고 있는) 블록
  - 어떤 디스크 블록을 할당할 것인가?



- 디스크 할당 방법
  - 연속 할당 (sequential allocation)
  - 불연속 할당 (non-sequential allocation)

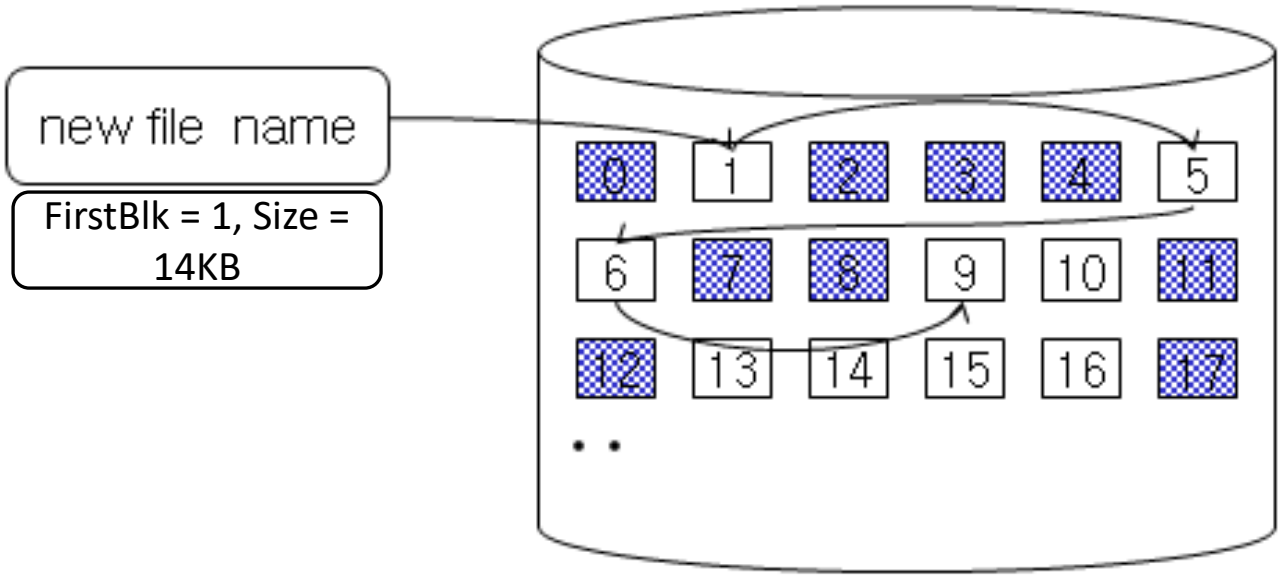
디스크의 블록 관리 (2/4)

- **연속 할당**



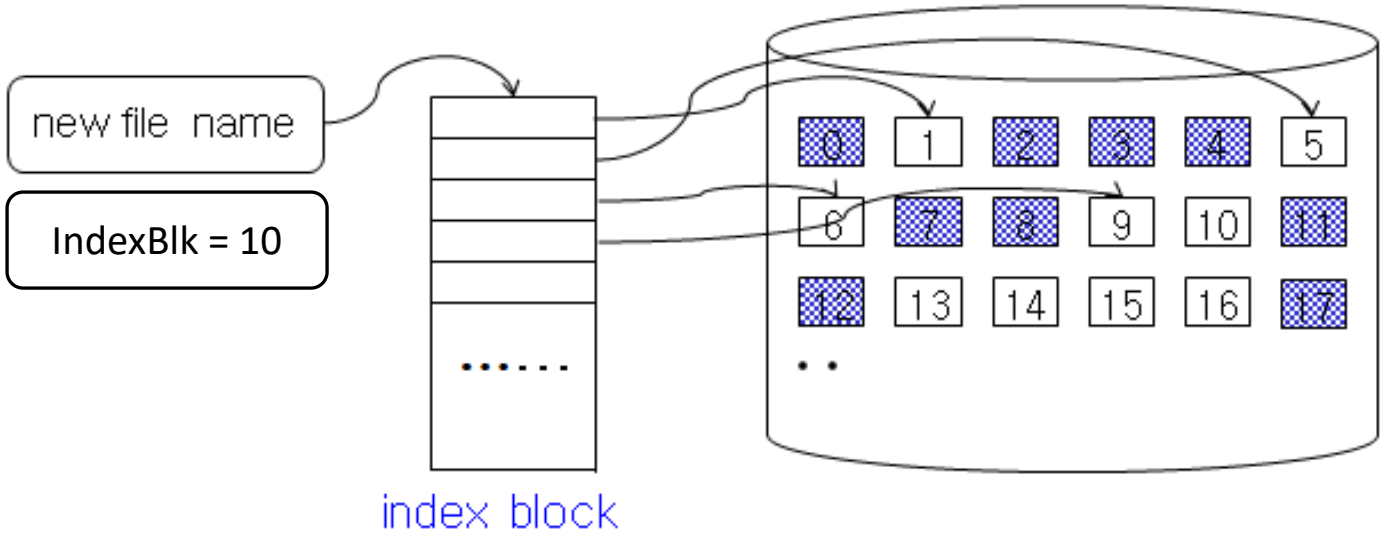
디스크 블록 관리 (3/4)

- **불연속 할당: 블록 체인 (block chain) 기법**
  - 현재 시나리오에서는 1, 5, 6, 9 블록을 할당
  - 할당된 블록을 링크로 연결
    - 링크를 가리키는 별도의 테이블 필요



디스크 블록 관리 (4/4)

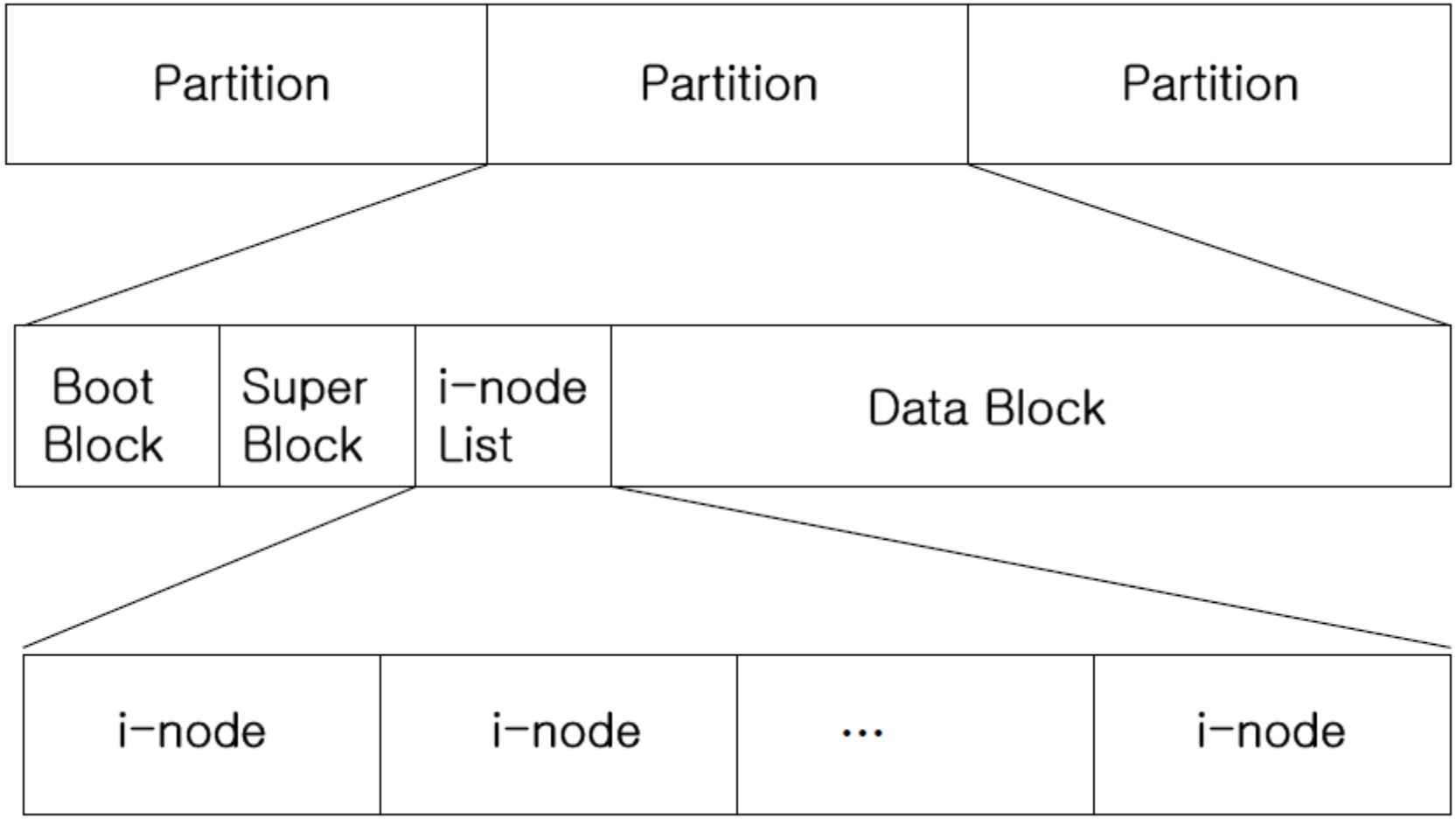
- **불연속 할당: 인덱스 블록 (index block) 기법**



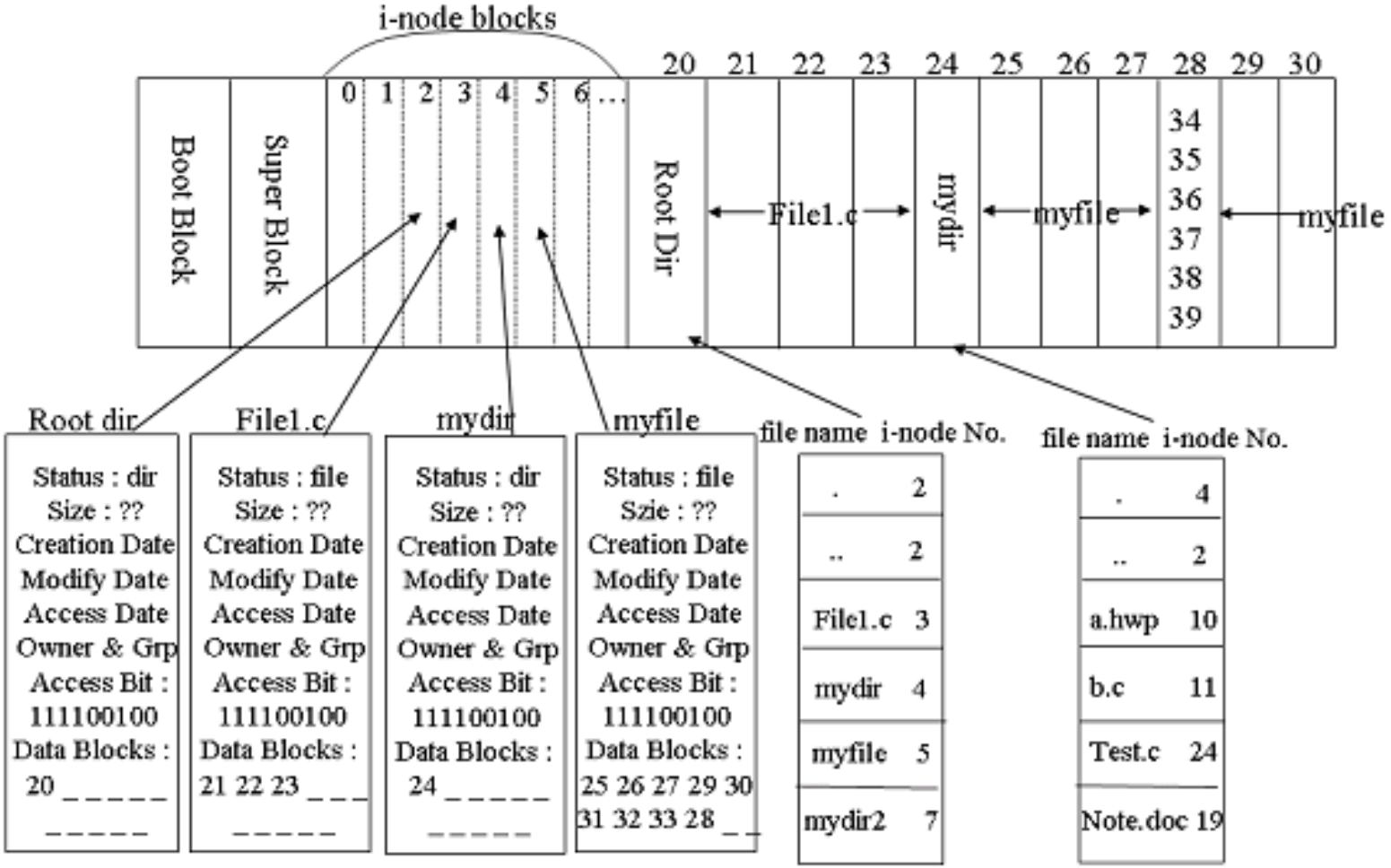
☞ 인덱스 블록이 꽉 차면????

Linux 파일 시스템 개념

- 파일 시스템 구조



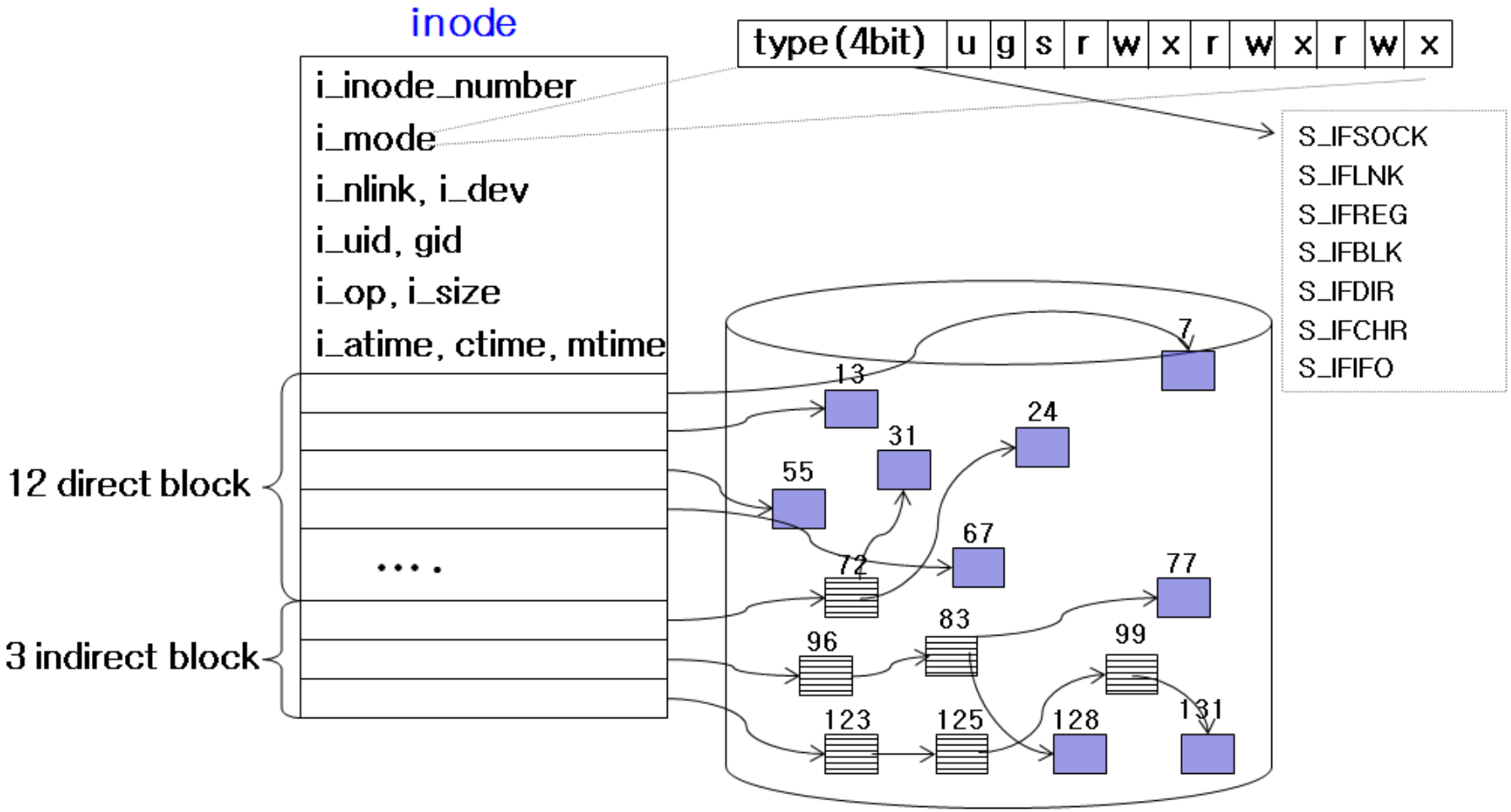
UNIX 파일 시스템 구조



inode (1/2)

- **inode 정의**
  - Linux에서 파일을 관리하기 위한 객체
    - 파일 하나당 하나의 inode를 가짐
    - 파일이 생성될 때 만들어짐
  - 파일의 모든 정보를 관리
    - 파일의 속한 위치 (index block과 유사)
    - 파일 소유자 및 권한
    - 파일 시간 정보
    - 파일 유형
      - 커널은 정규 파일뿐만 아니라 디렉토리, 디바이스, 파이프 소켓 등도 파일이라는 추상화 객체로 관리

inode (2/2)


inode
i_inode_number
i_mode
i_nlink, i_dev
i_uid, gid
i_op, i_size
i_atime, ctime, mtime
12 direct block
3 indirect block
type (4bit)
u g s r w x r w x r w x
S_IFSOCK
S_IFLNK
S_IFREG
S_IFBLK
S_IFDIR
S_IFCHR
S_IFIFO
7
13
24
31
55
67
72
77
83
96
99
123
125
128
131

Linux 파일 시스템 개념

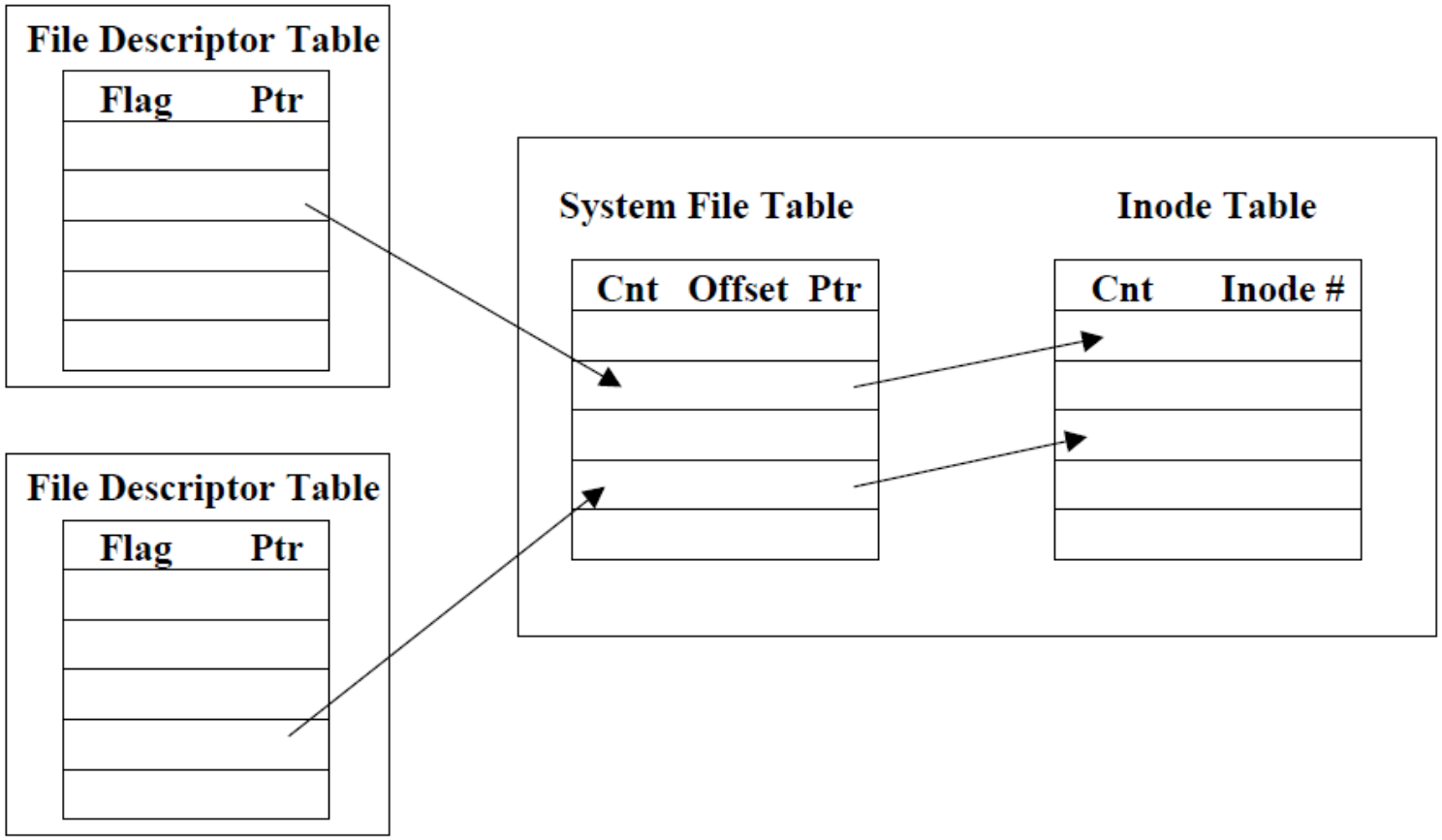
- **Superblock**
  - 파일 시스템 크기
  - 파일 시스템의 자유 블록의 수
  - 파일 시스템에서 사용 가능한 자유 블록 리스트
  - inode 리스트의 크기
  - 파일 시스템에서 사용 가능한 inode 수
  - 파일 시스템내의 사용 가능한 inode 리스트

Linux 파일 시스템 개념

- **inode list**
  - 각 항은 하나의 파일과 대응
  - 파일 오픈시 디스크에 있는 inode 정보를 메모리로 읽어옴
    - In-core inode라고 불림
    - 참조개수, inode 번호, 파일 시스템 장치 번호

Linux 파일 시스템 개념

- **File descriptor table, system file table, inode table**



Linux 파일 시스템 개념

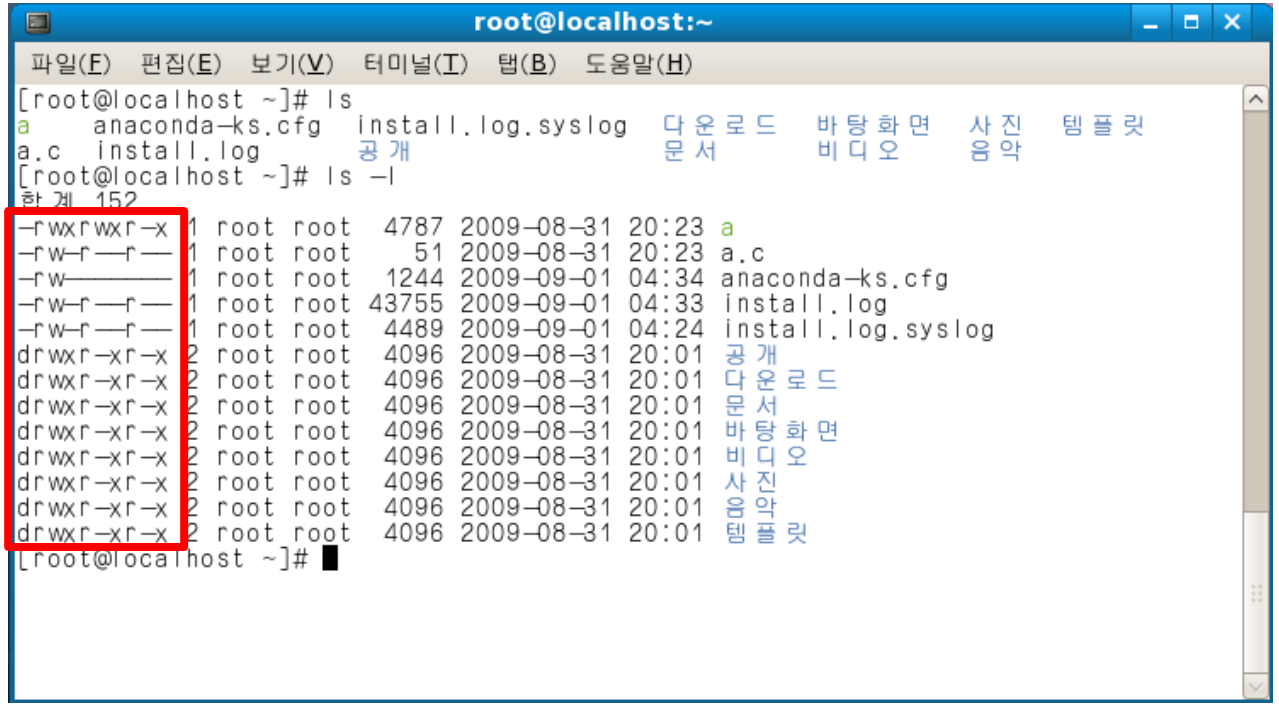
- **File descriptor table**
  - 개별 프로세스 마다 존재 (프로세스와 같은 lifetime)
  - 항목 구성
    - File descriptor
    - Flag
    - File table pointer
- **System file table**
  - 파일 시스템에 하나
  - 항목 구성
    - 참조 계수 (Reference count)
    - 파일의 현재 offset
    - inode table을 가리키는 포인터
    - 파일 상태 flag
- **Inode table**
  - In-core inode를 의미

파일 입출력

유닉스 프로그래밍

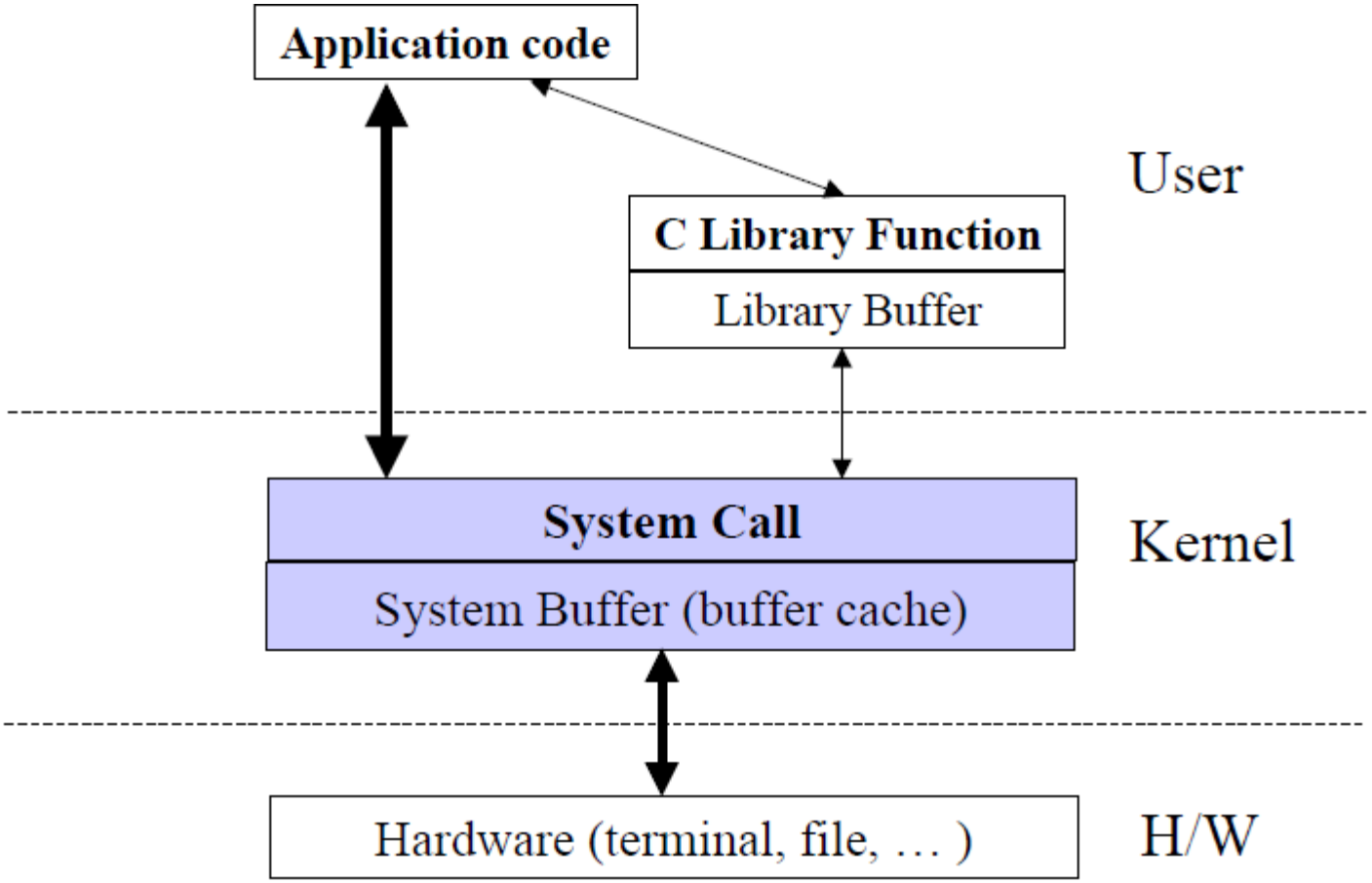
System Call I/O

리눅스 파일의 허가 모드



- **USER/GROUP/OTHER로 권한이 나뉨**
  - r: 읽기 권한
  - w: 쓰기 권한
  - x: 실행 권한

System Call vs. Standard Library



에러 처리

NAME

perror,strerror

SYNOPSIS

```
#include <string.h>
char *strerror( int  errnum);

#include <stdio.h>
void  perror (const char *msg);
```

RETURN VALUE

Return : file descriptor if OK, - on error

에러 처리

- **Description**
  - 리눅스 함수는 일반적으로 음수를 리턴하여 에러가 발생했음을 알림
  - 에러 발생 시, 에러의 종류에 따라 전역 변수의 errno에 특정 값이 설정됨
  - <error.h>에 에러의 종류와 해당 값이 정의되어 있음

strerror

- 전달받은 오류 번호에 해당하는 오류 메시지 문자열의 주소를 반환 (errno를 인수로 전달하여 사용)

perror

- 현재의 errno값을 읽어 해당하는 오류 메시지를 msg와 함께 표준 오류(stderr)로 출력

open function

NAME

open - open or create a file or device

SYNOPSIS

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int open ( const char *pathname, int oflag);
int open ( const char *pathname, int oflag, mode_t  mode);
```

RETURN VALUE

Return : file descriptor if OK, -1 on error

open function

- **Description**
  - oflag – open의 목적
    - O_RDONLY: 읽기 전용
    - O_WRONLY: 쓰기 전용
    - O_RDWR: 읽기, 쓰기 가능

    - O_APPEND: 모든 쓰기 작업은 파일의 끝에서 수행된다.
    - O_CREAT: 파일이 없을 경우 파일 생성
    - O_DIRECT: 직접 입출력 수행 위해 파일을 연다.
    - O_EXCL: O_CREAT시 파일이 있는 경우에 error 발생
    - O_TRUNC: 파일이 있는 경우에 기존 파일을 지운다.
    - O_NONBLOCK: blocking I/O를 nonblocking 모드로 바꿈
    - O_SYNC: 매 쓰기 연산마다 디스크 I/O가 발생하도록 설정

  - mode – 파일의 접근권한
    - 새로운 파일 생성시에만 사용(O_CREAT)

open function

open.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        char *fname = "test.txt";
        int fd;

        fd = open(fname, O_RDONLY);
        if( fd >= 0 )
        {
                printf("Success\n FIleName: %s, fd: %d\n", fname, fd);
                close(fd);
        }
        else
                perror("open failed\n");


        return 0;
}
```

creat function

NAME

creat - create a file or device

SYNOPSIS

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int creat ( const char *pathname, mode_t mode );
```

RETURN VALUE

Return : file descriptor if OK, -1 on error

creat function

- **Description**
  - pathname – 생성 파일 경로
  - mode – 생성 파일의 허가 모드
    - umask값과 연산하여 permission 결정
  - default로 write only로 생성 됨 (Read 필요 시 파일을 다시 open 해야 함)
  - open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode)와 동등
  - ※umask shell command로 현재 umask 확인 가능
  - ※defualt umask 값은 /etc/bashrc에서 확인 가능
    umask 초기 설정 위치는 배포판과 셸 설정에 따라 다름

creat function

creat.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        char *fname = "test.txt";
        int fd;

        fd = creat(fname, 0666);
        if( fd < 0 )
                perror("create()");
        else
        {
                printf("Success!\n Filename: %s, fd : %d\n", fname, fd);
                close(fd);
        }

        return 0;
}
```

close function

NAME

close - close a file descriptor

SYNOPSIS

```
#include <unistd.h>
int close(int fd);
```

RETURN VALUE

Return : 0 if OK, -1 on error

read function

NAME

read - read data from an open file

SYNOPSIS

```
#include <unistd.h>
ssize_t  read (int filedes,  void *buf, size_t nbytes);
```

RETURN VALUE

Return : number of bytes read, 0 if end of file, -1 on error

read function

- **Description**
  - filedes – open 파일 디스크립터 번호
  - buffer – 파일로부터 데이터를 읽어 들일 메모리 주소
  - nbytes – 읽어 들일 데이터의 최대 byte 수

  - Return 값에 따른 대응 필요
    - return값 == nbytes: 성공
    - 0 < return 값 < nbytes
      - 파일 끝(EOF)에 도달하는 등의 이유로 요청량보다 적게 읽을 수 있음
    - return 값 == 0: 파일 끝(EOF)에 도달
    - 리턴 안됨
      - 디바이스 파일의 경우 데이터가 입력될 때까지 프로세스 중단
      - 단, 시그널이 오면 깨어나서 리턴, return 값 == -1
    - return 값 == -1
      - 시그널로 읽기 중단: errno == EINTR
      - NON-BLOCKING 모드인 경우: errno == EAGAIN
      - 읽는 도중 오류가 발생: errno가 EINTR이나 EAGAIN이 아닌 다른 값. 심각한 오류

read function

read1.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        int fd, linecnt = 0, cnt;
        char c;
        char *fname = "test.txt";

        if( (fd = open(fname, O_RDONLY)) < 0 )
        {
                perror("open()");
                exit(-1);
        }
        while ( (cnt  = read(fd, &c, 1)) > 0 ) {
                /* Here, cnt == 1 */
                if( c=='\n') linecnt++;
        }
        if (cnt== -1) {
                perror(“read”);
        }
        printf("Total line: %d\n", linecnt);
        close(fd);
        return 0;
}
```

read function: 여러바이트 읽기

read2.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>
#define BUFSIZE  2

int main(void)
{
        int fd, linecnt = 0, cnt,  i;
        char buf[BUFSIZE];
        char *fname = "test.txt";

        if( (fd = open(fname, O_RDONLY)) < 0 )
        {
                perror("open()");
                exit(-1);
        }
        while ( (cnt  = read(fd, buf, BUFSIZE)) > 0 ) {
                for (i = 0; i < cnt; i++)  {
                    if( buf[i]=='\n') linecnt++;
                }
        }
        if (cnt== -1) {
                perror(“read”);
       }

        printf("Total line: %d\n", linecnt);
       close(fd);
        return 0;
}
```

Non-blocking read

read3.c

```
#include <stdio.h>
#include <fcntl.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)  {
        int  fd, nr;
        char  c,  *fname = "/dev/tty";

        if( (fd = open(fname, O_RDONLY | O_NONBLOCK)) < 0 )   {
                perror("open()");
                exit(-1);
        }
        for( ; ; ) {
                if ((nr =  read(fd, &c, 1)) > 0 ) {
                        printf("%c\n", c);
                        exit(0);
                }
                else if (nr == -1) {
                        if (errno == EINTR) printf(".");
                        if (errno == EAGAIN) printf("*");
                }
                else  break;
        }
         close(fd);
}
```

터미널의 정규 모드(canonical mode)에서는 Enter를 눌러야
입력한 줄이 프로세스에 전달됨

괄호 개수 주의

write function

NAME

write - write data from an open file

SYNOPSIS

```
#include <unistd.h>

ssize_t  write (int filedes,  const void *buf, size_t nbytes);
```

RETURN VALUE

Return : number of bytes written if OK, -1 on error

write function

- **Description**
  - filedesc – open 파일 디스크립터
  - buffer – 데이터가 들어있는 메모리 주소
  - nbytes – 기록할 데이터의 byte 수

  - Return 값과 nbytes 값은 같아야 한다. (다르면 에러)

Read/write 예제: copy.c

copy.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
        int fd1, fd2, n;
        char  buf[100];

        if (argc < 3) {
                printf(" Usage:  copy   oldfile   newfile \n");    exit(0);
        }
        if( (fd1= open(argv[1], O_RDONLY)) < 0 ) {
                perror("open() ");       exit(-1);
        }
        fd2 = open(argv[2], O_WRONLY|O_CREAT | O_TRUNC, 0666);
        if (fd2 < 0) {
                perror("open() ");       exit(-1);
        }
        while ((n = read(fd1, buf, 100)) > 0 )
                write(fd2, buf, n);
        close(fd1);
        close(fd2);
        return 0;
}
```

```
# ./copy   a.c    b.c
argc = 3
argv[0] = "./copy"
argv[1] = "a.c"
argv[2] = "b.c"
```

lseek function

NAME

lseek - reposition read/write file offset

SYNOPSIS

```
#include <sys/types.h>
#include <unistd.h>
off_t lseek(int fildes, off_t offset, int whence);
```

RETURN VALUE

Return : new file offset if OK, -1 on error

lseek function

- **Description**
  - offset – 기준점에서의 상대적 거리
    - 음수도 가능 (SEEK_CUR, SEEK_END)
  - whence – offset 지정시의 기준점
    - SEEK_SET: 파일의 시작
    - SEEK_CUR: 파일의 현재
    - SEEK_END: 파일의 끝
  - 파일의 offset
    - System file table에 위치함
    - 파일을 처음부터 byte 단위로 계산한 정수 값
    - 기본적으로 파일이 열리면 offset의 위치는 0 (O_APPEND 예외)
    - 파일의 크기보다 큰 값으로 설정하면?

lseek function

lseek.c

```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(void)
{
        char *fname = "test.txt";
        int fd;
        off_t fsize;

        if( (fd = open(fname, O_RDONLY)) < 0 )
        {
                perror("open()");
                exit(-1);
        }
        fsize = lseek(fd, 0, SEEK_END);
        printf("The size of <%s> is %lu bytes.\n", fname, fsize);
        return 0;
}
```

Process vs. File Descriptor (1)

- **한 Process가 동일 파일을 2번 open하여 read???**

Process vs. File Descriptor (1)

mread.c

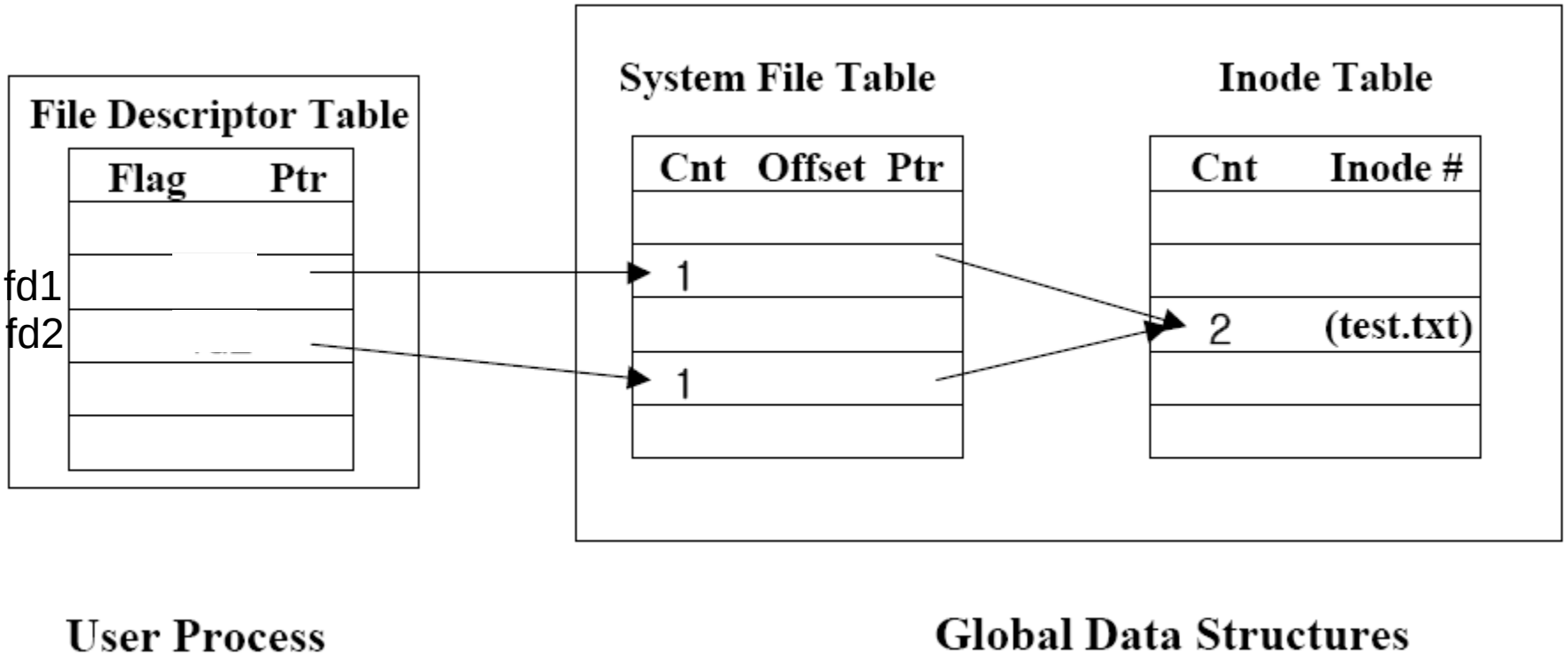
```
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

int main(void)
{
        char *fname = "test.txt";
        int fd1, fd2, cnt;
        char buf[30];

        fd1 = open(fname, O_RDONLY);
        fd2 = open(fname, O_RDONLY);
        if( fd1 < 0 || fd2 < 0 ) {
                perror("open()");
                exit(-1);
        }
        cnt = read(fd1, buf, 12);
        buf[cnt] = '\0';
        printf("fd1's first printf: %s\n", buf);
        lseek(fd1, 1, SEEK_CUR);
        cnt = read(fd1, buf, 12);
        buf[cnt] = '\0';
        printf("fd1's second printf: %s\n", buf);
        cnt = read(fd2, buf, 12);
        buf[cnt] = '\0';
        printf("fd2's first printf: %s\n", buf);
        lseek(fd2, 1, SEEK_CUR);
        cnt = read(fd2, buf, 12);
        buf[cnt] = '\0';
        printf("fd2's second printf: %s\n", buf);
        return 0;
}
```

Process vs. File Descriptor (1)

- **File Descriptor Table, System File Table, Inode Table**



Process vs. File Descriptor (1)

- **서로 다른 Process가 동일 파일을 open하여 read???**

Process vs. File Descriptor (2)

readA.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
   int fd;
   char c;

   if ( (fd = open("test.txt", O_RDONLY)) < 0 )
   {
            perror("open");
            exit(0);
   }
   while (read(fd, &c, 1) > 0 ) {
            putchar(c);
   }
   read(0, &c, 1);
   return 0;
}
```

readB.c

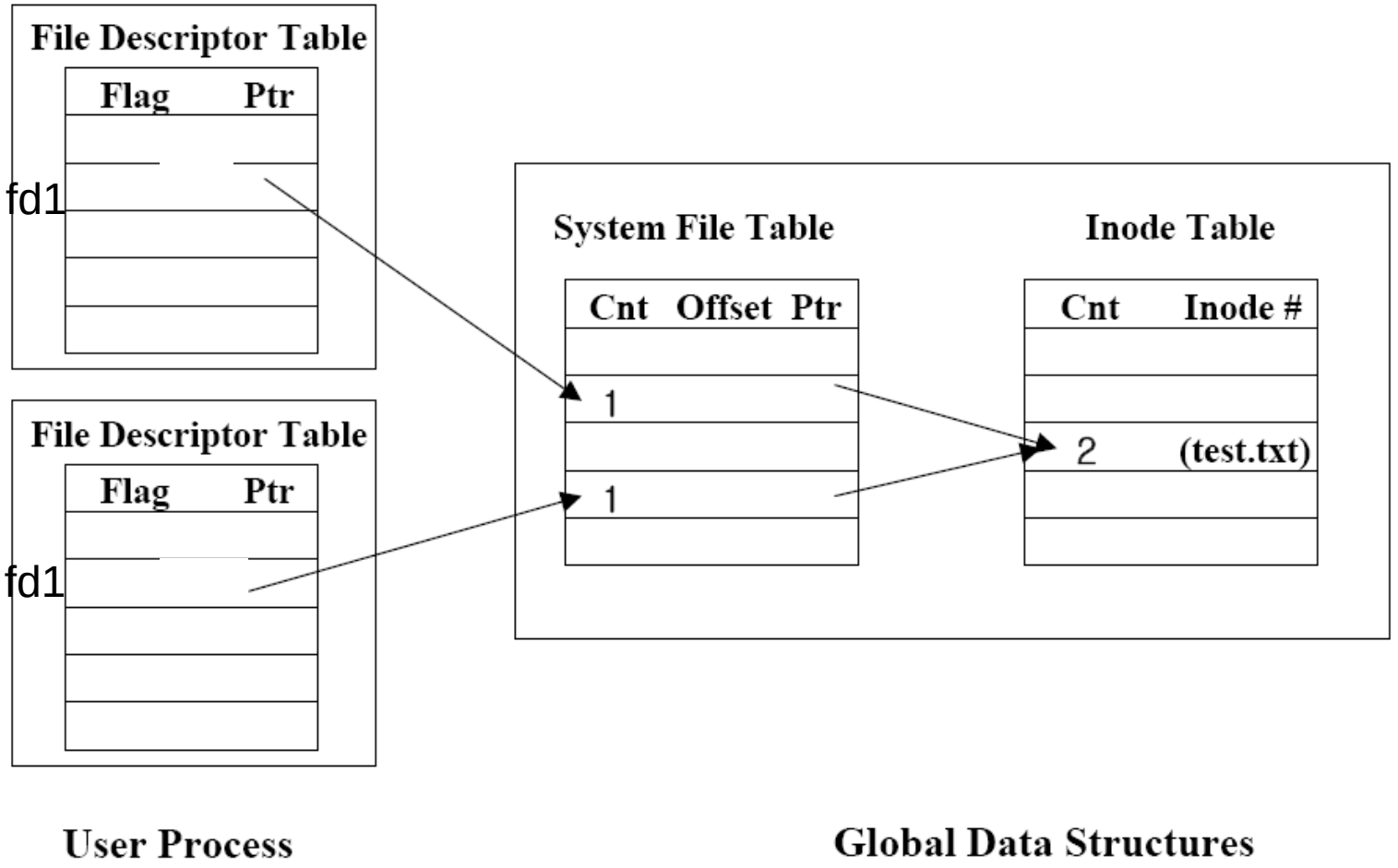
```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
   int fd, linecnt = 0;
   char c;
   char *fname = "test.txt";

   if ( (fd = open(fname, O_RDONLY)) < 0 ) {
            perror("open()");
            exit(-1);
   }
   while ( read(fd, &c, 1) > 0 ) {
            putchar(c);
            if( c == '\n' )
                     linecnt++;
   }
   printf("Total line: %d\n", linecnt);
   read(0, &c, 1);
   return 0;
}
```

Process vs. File Descriptor (1)

- **File Descriptor Table, System File Table, Inode Table**



pread, pwrite

NAME

pread, pwrite

SYNOPSIS

```
#define _XOPEN_SOURCE 500
#include <unistd.h>

ssize_t pread (int fd, void *buf, size_t count, off_t  pos);
ssize_t pwrite (int fd, void *buf, size_t count, off_t  pos);
```

RETURN VALUE

Return: number of bytes read or written, 0 if EOF in pread,
0 if nothing written in pwrite, -1 if error

fsync, fdatasync, sync

NAME

fsync, fdatasync

SYNOPSIS

```
#include <unistd.h>

int fsync (int fd);
int fdatasync (int fd);
void sync (void);
```

RETURN VALUE

Return: 0 if OK, -1 if error

fsync, fdatasync, sync

- **Description**
  - fsync(fd) : fd의 메타데이타와 파일 데이터 저장
  - fdsync(fd): fd의 파일 데이터 저장
  - sync() : 시스템의 모든 변경된 데이터 저장

O_SYNC Flag

wsync.c

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        char *fname = "test.txt";
        int    fd;
        char *c, *buf = "This is test";

        fd = open(fname, O_WRONLY | O_SYNC);
        if( fd < 0 ) {
                perror("open()");
                exit(-1);
        }
        for (c = buf; *c != '\0'; c++) {
                write(fd, c, 1);
        }
        close(fd);
        return 0;
}
```

ftruncate, truncate

NAME

ftruncate, truncate

SYNOPSIS

```
#include <unistd.h>
#include <sys/types.h>

int ftruncate (int fd, off_t  len);
int truncate (const char *path, off_t  len);
```

RETURN VALUE

Return: 0 if OK, -1 if error

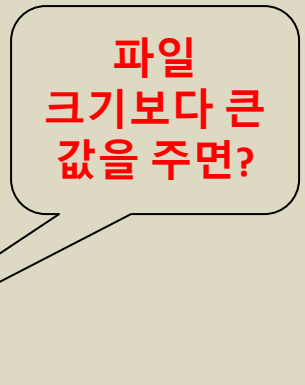
truncate example

truncate.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
        int    ret;

        ret = truncate ("./test.txt", 8);
        if (ret == -1) {
                perror("truncate");
                return -1;
        }
        return 0;
}
```



dup, dup2

NAME

dup,dup2 - duplicate a file descriptor

SYNOPSIS

#include <unistd.h>

int dup (int *filedes*);

int dup2 (int *filedes* , int *newfiledes*);

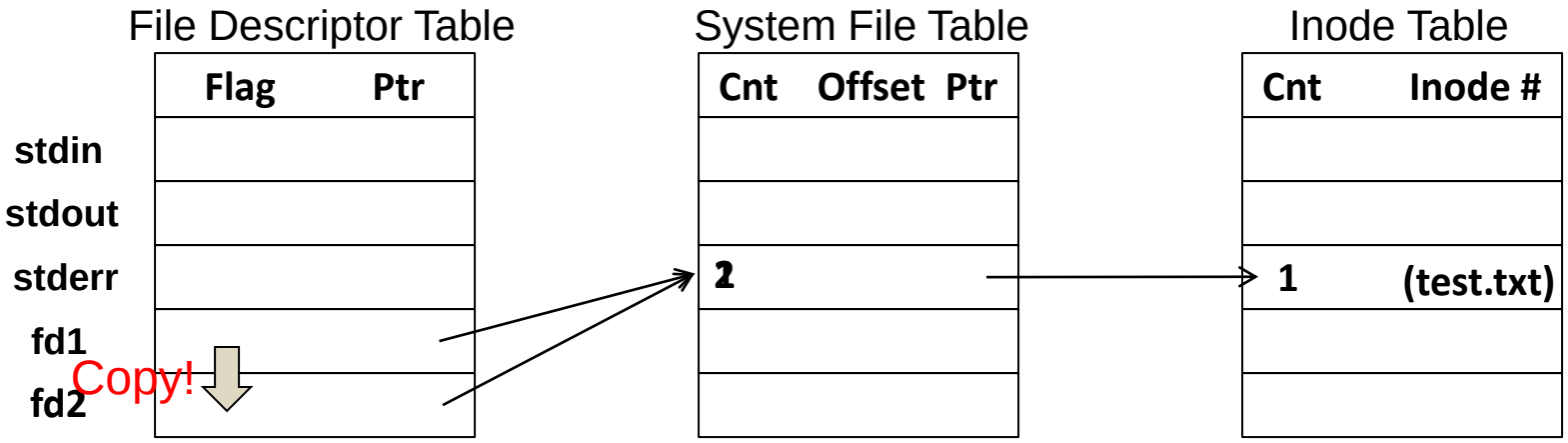
RETURN VALUE

Return : new file descriptor if OK, -1 on error

dup, dup2

- **Description**
  - File descriptor를 복사
  - dup(int filedes)
    - 인자로 넘어온 file descriptor의 엔트리를 사용되지 않는 가장 작은 file descriptor table의 엔트리로 복사

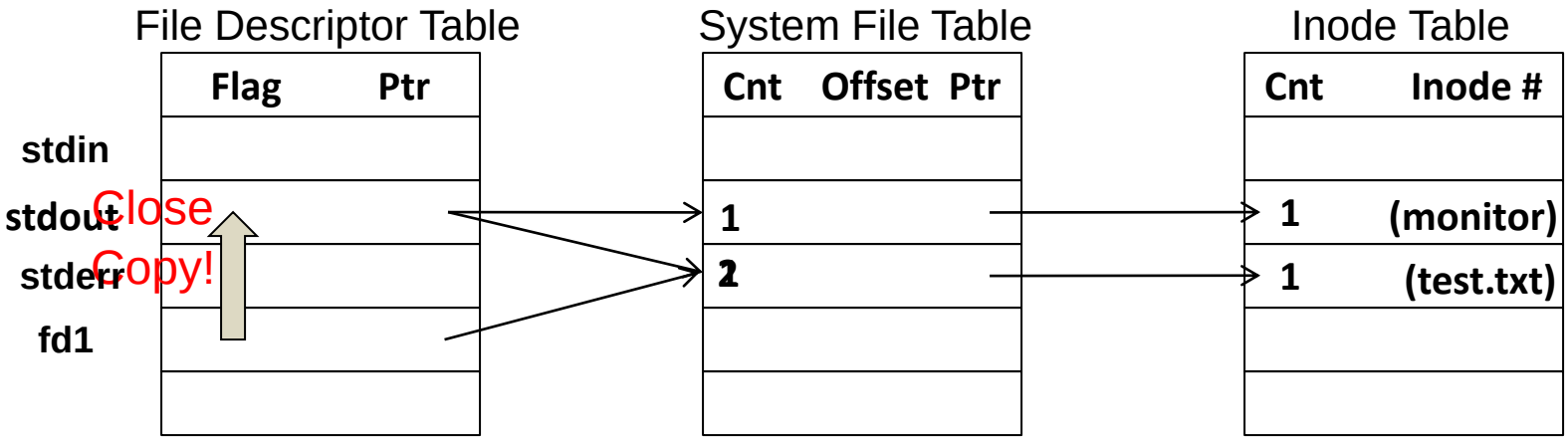
fd2 = dup(fd1)을 수행했을 때



dup, dup2

- **Description (Cont'd)**
  - dup2(filedes, newfiledes)
    - filedes가 가리키는 file descriptor의 엔트리를 newfiledes가 가리키는 곳으로 복사
    - 만약 newfiledes가 가리키는 곳에 이미 엔트리가 존재할 경우 닫고 할당

dup2(fd1, 1)을 수행했을 때



dup

dup1.c

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        char *fname = "test.txt";
        int fd1, fd2, cnt;
        char buf[30];

        fd1 = open(fname, O_RDONLY);
        if( fd1 < 0 ) {
                perror("open()");
                exit(-1);
        }
        fd2 = dup(fd1);
        cnt = read(fd1, buf, 12);
        buf[cnt] = '\0';
        printf("fd1's printf: %s\n", buf);
        lseek(fd1, 1, SEEK_CUR);
        cnt = read(fd2, buf, 12);
        buf[cnt] = '\0';
        printf("fd2's printf: %s\n", buf);
        return 0;
}
```

dup2

dup2.c

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
        char *fname = "test.txt";
        int fd;

        if( (fd = creat(fname, 0666)) < 0 ) {
                perror("creat()");
                exit(-1);
        }

        printf("First printf is on the screen.\n");
        dup2(fd, 1);
        printf("Second printf is in this file.\n");

        return 0;
}
```

Multiplexed I/O: select

NAME

select

SYNOPSIS

```
#include <sys/time.h>
#include <sys/type.h>
#include <unistd.h>

int select (int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
            struct timeval *timeout);
FD_CLR (int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
FD_SET (int fd, fd_set *set);
FD_ZERO (fd_set, *set);
```

RETURN VALUE

Return: number of fds ready for I/O, 0 if timeout, -1 if error

select example

sel.c

```
# gcc  –o  sel  sel.c
# sudo  ./sel
```

```
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

#define TIMEOUT 5
#define BUF_LEN 1024

int main(void)
{
   struct  timeval  tv;
   fd_set  readfds;
   int  ret, mfd;

   mfd = open("/dev/input/mouse0", O_RDONLY);
   if (mfd < 0) {
         printf("mouse device open fail\n");
         return 1;
   }

   FD_ZERO(&readfds);
   FD_SET(STDIN_FILENO, &readfds);
   FD_SET(mfd, &readfds);

   tv.tv_sec = TIMEOUT;
   tv.tv_usec = 0;
```

```
   ret = select(mfd+1, &readfds,
                NULL, NULL, &tv);
   if (ret == -1) return 1; /* select error */
   else if (!ret) return 0; /*timeout */

   if (FD_ISSET(STDIN_FILENO, &readfds)) {
      char buf[BUF_LEN+1];
      int len;

      len = read(STDIN_FILENO, buf, BUF_LEN);
      if (len == -1) return 1;  /* read error */
      if (len) {
         buf[len] = '0';
         printf("read: %s\n", buf);
      }
      return 0;
   }
   if (FD_ISSET(mfd, &readfds)) {
       printf("mouse event occurred\n");
       return 0;
   }
   fprintf(stderr, "This should not happen!\n");
   return 1;
}
```

select example 2

selfifo.c

mkfifo np1 np2

```
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

#define TIMEOUT 5
#define BUF_LEN 1024

int main(void)
{
   struct  timeval  tv;
   fd_set  readfds;
   int  ret, fd1, fd2;

   fd1 = open ("./np1", O_RDONLY);
   if (fd1 < 0) {
         printf("pipe file 1 open fail\n");
         return 1;
   }
   fd2 = open("./np2", O_RDONLY);
   if (fd2 < 0) {
         printf("pipe file 2 open fail\n");
         return 1;
   }
```

```
   FD_ZERO(&readfds);
   FD_SET(fd1, &readfds);
   FD_SET(fd2, &readfds);

   tv.tv_sec = TIMEOUT;
   tv.tv_usec = 0;

   ret = select(fd2+1, &readfds,
                  NULL, NULL, &tv);
   if (ret == -1) return 1; /* select error */
   else if (!ret) return 0; /*timeout */

   if (FD_ISSET(fd1, &readfds)) {
      printf("pipe file 1 data read\n");
      return 0;
   }
   if (FD_ISSET(fd2, &readfds)) {
       printf("pipe file 2 data read\n");
       return 0;
   }
   fprintf(stderr, "This should not happen!\n");
   return 1;
}
```

Multiplexed I/O: pselect

NAME

select

SYNOPSIS

```
#define _XOPEN_SOURCE 600
#include <sys/select.h>

int pselect (int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
             const struct timespec*timeout, const sigset_t *sigmask);
FD_CLR (int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
FD_SET (int fd, fd_set *set);
FD_ZERO (fd_set, *set);
```

RETURN VALUE

Return: number of fds ready for I/O, 0 if timeout, -1 if error

Multiplexed I/O: poll

NAME

poll

SYNOPSIS

```
#include <sys/poll.h>

int poll (struct pollfd *fds, unsigned int nfds, int timeout);

struct pollfd {
    int fd;
    short events;
    short revents;
}
```

RETURN VALUE

Return: number of fds ready for I/O, 0 if timeout, -1 if error

Multiplexed I/O: ppoll (Linux Only)

NAME

poll

SYNOPSIS

```
#include <sys/poll.h>

int ppoll (struct pollfd *fds, unsigned int nfds, int timeout,
           const sigset_t *sigmask);

struct pollfd {
    int fd;
    short events;
    short revents;
}
```

RETURN VALUE

Return: number of fds ready for I/O, 0 if timeout, -1 if error

poll example

poll.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/poll.h>
#include <stdlib.h>


#define TIMEOUT 5

int main(void)
{
   struct  pollfd  fds[2];
   int  ret;

   fds[0].fd = STDIN_FILENO;
   fds[0].events = POLLIN;

   fds[1].fd = STDOUT_FILENO;
   fds[1].events = POLLOUT;
```

```
   ret = poll (fds, 2, TIMEOUT *1000);

   if (ret == -1) return 1; /* poll error */
   else if (!ret) return 0; /*timeout */

   if (fds[0].revents & POLLIN) {
      printf("stdin is readable\n");
   }

   if (fds[1].revents & POLLOUT) {
      printf("stdout is writable\n");
   }

   return 0;
}
```

poll example2

poll2.c

mkfifo np1 np2

```
#include <stdio.h>
#include <unistd.h>
#include <sys/poll.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/time.h>
#include <sys/types.h>

#define TIMEOUT 20

int main(void)
{
   struct  pollfd  fds[2];
   int  ret, fd1, fd2;

   fd1 = open ("./np1", O_RDONLY);
   if (fd1 < 0) {
         printf("pipe file 1 open fail\n");
         return 1;
   }
   fd2 = open("./np2", O_RDONLY);
   if (fd2 < 0) {
         printf("pipe file 2 open fail\n");
         return 1;
   }
```

```
   fds[0].fd = fd1;
   fds[0].events = POLLIN;

   fds[1].fd = fd2;
   fds[1].events = POLLIN;

   ret = poll (fds, 2, TIMEOUT *1000);

   if (ret == -1) return 1; /* poll error */
   else if (!ret) return 0; /*timeout */

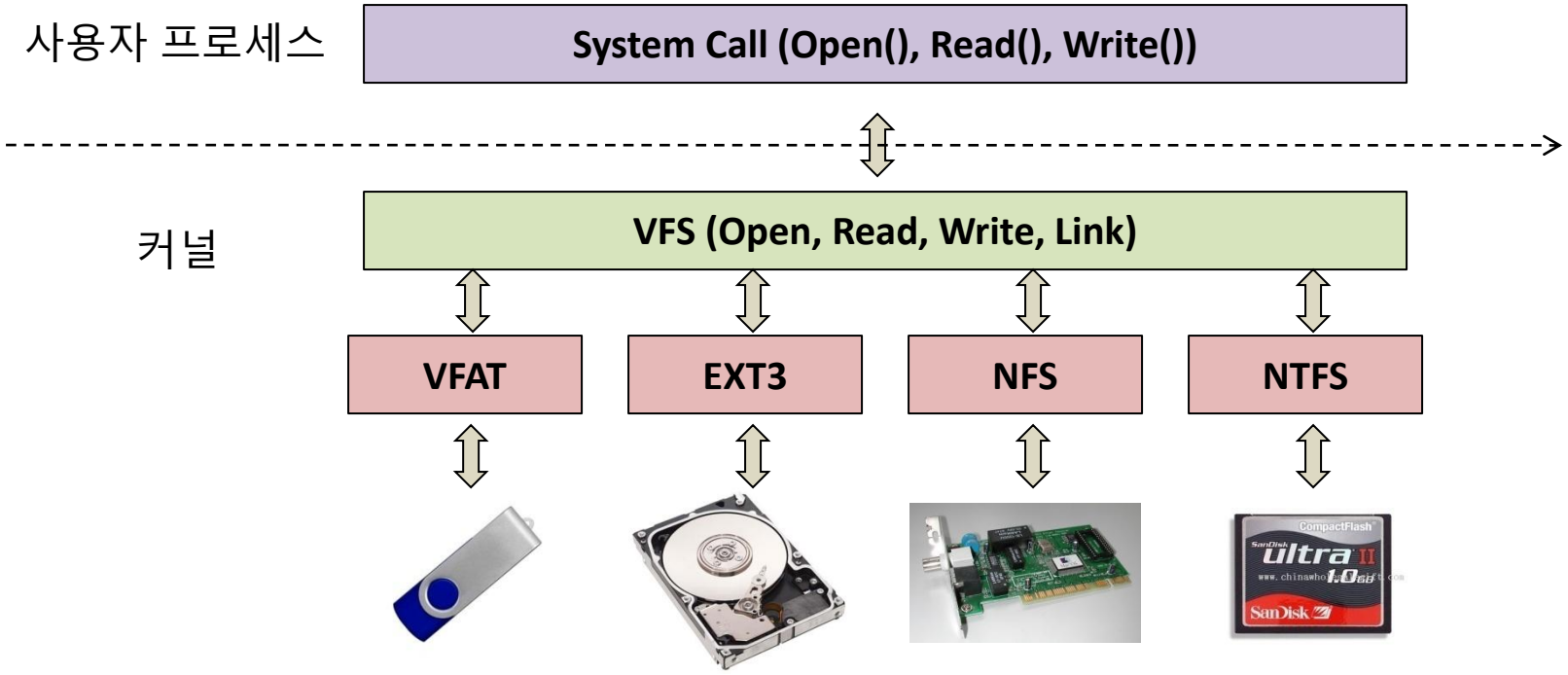
   if (fds[0].revents & POLLIN) {
      printf("np1 is readable\n");
   }
   if (fds[1].revents & POLLIN) {
      printf("np2 is readable\n");
   }
   return 0;
}
```

커널 내부

- **Virtual File System**
- **Page cache**
- **Page Writeback**

Virtual File System

- **공통 파일 모델 (Common File Model)**
  - 리눅스에서 모든 파일 시스템이 따라야 하는 framework
    - 읽기, 쓰기, 링크 생성 등 파일 시스템이 정의해야 하는 인터페이스 제공



Virtual File System

- **장점**
  - 한가지의 시스템 호출 통해 어떤 매체에 있는 파일시스템도 읽을 수 있음
  - 유틸리티 하나로 서로 다른 파일시스템 사이에 복사 지원 가능 → 읽기, 쓰기 가능
  - 모든 파일시스템은 동일한 개념, 동일한 인터페이스, 동일한 호출 지원
  - 새로운 파일시스템을 개발하더라도 리눅스에서 쉽게 사용 가능
    - ???

Page Cache

- **Locality of reference**
  - Spatial locality
    - 현재 참조되는 자원과 인접한 곳에서 참조가 발생할 확률이 높음
  - Temporal locality
    - 특점 시점에서 자원에 접근되면 그 자원은 다시 참조될 확률이 높음

```
int num[10000], sum = 0, i;

……


for( i = 0; i < 10000; i++ )
   sum += num[i];

……
```

Page Cache

- **보조기억장치 (e.g. HDD, usb drive)와 메인 메모리 사이의 속도차를 완화 시키기 위함**
  - 가까운 미래에 참조될 것으로 판단되는 내용을 메인 메모리에 위치 시킴

- **리눅스 페이지 캐시**
  - 동적인 크기를 지원
    - 적은 영역을 사용하다 점진적으로 증가하다가 최대 크기가 되었을 때 멈춤
    - 최대 크기가 되었을 때, 어떤 내용을 교체해야 할까?
  - 대표적인 교체정책
    - FIFO (First in first out), RANDOM, LRU (Least Recently Used), LFU (Least Frequently Used)

- **Hit ratio**
  - cache hit count / total reference count
  - 높을 수록 좋은 캐시 교체 정책 알고리즘

- **Read-ahead (prefetching)**
  - 앞으로 접근될 것으로 판단되는 자원을 미리 메인 메모리로 읽어 놓음
  - 미리 정해놓은 크기가 될 때 까지, 읽어오는 크기를 점진적으로 크기를 증가 시킴

Page Writeback

- **페이지 캐시에 존재하는 내용 중 쓰인 내용 (dirty page)을 다시 보조기억장치에 적는 일**

- **Page Writeback이 발생하는 상황**
  - 미리 정의된 메모리 양 보다 페이지 캐시의 여유 메모리 공간이 줄어들 때
  - 미리 정의된 시간보다 변경된 내용이 오래되었을 때

- **pdflush (Page Dirty FLUSH)**
  - 위의 두 조건 중 하나라도 만족되면 이 커널스레드가 깨어나 page writeback 수행

- **pdflush를 통해 데이터를 적기 전에 시스템이 꺼지면?**